Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University



ME701 Embedded Systems 1

Hardware Interfaces & Control Part 2

Dr John Wade
Email: john.wade@atu.ie

Overview

- **What is a Stepper motor?**
  - Why use them, typical Applications
  - Advantages / Disadvantages
- **Different Types of Stepper Motor**
- **Interfacing with Stepper Motors**
  - Drivers
  - Code for interfacing and control (MSP430)
- **How stepper motors operate.**
  - Different Modes of Operation
- **What are Servo Motors?**
  - Operation and Control
  - Code for interfacing and control (MSP430)





Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Interfacing to Stepper Motors

What are Stepper Motors

- **What is a stepper motor?**
  - A stepper motor is an electromechanical device that converts electrical pulses into discrete mechanical movements
  - It moves in steps or discrete angular movements, with each pulse sent to the motor resulting in a specific degree of rotation
  - Stepper motors are used in applications where precise positioning and speed control are required

Common Applications

- Common applications include:
  - Robotics – for precise joint control
  - 3D printers – to control the position of the print head
  - CNC machines – to move tools with high precision
  - Camera focus control – for adjusting focus smoothly

Why Use Stepper Motors?

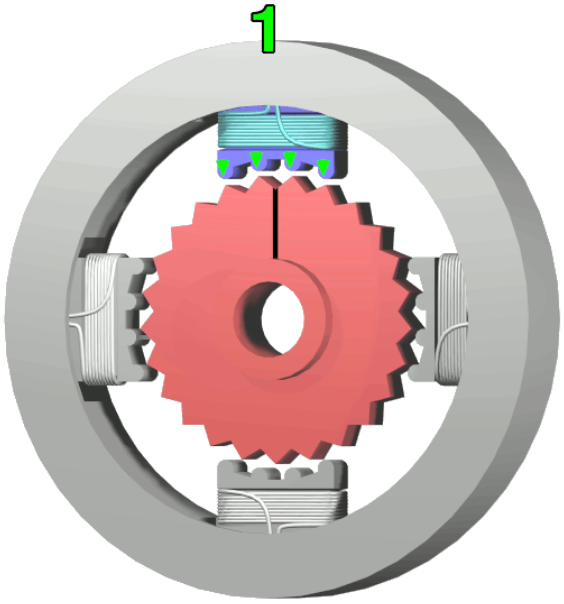
- **Advantages:**
  - Precise positioning without feedback (open-loop control)
  - High torque at low speeds, making them suitable for applications that require holding force
  - Repeatable accuracy, as each step is consistent with the previous ones
- **Limitations:**
  - Lower efficiency compared to other motor types like DC motors
  - Prone to step losses or missed steps, especially at high speeds
  - Requires a driver circuit to control the step sequence

Stepper Motor Basics

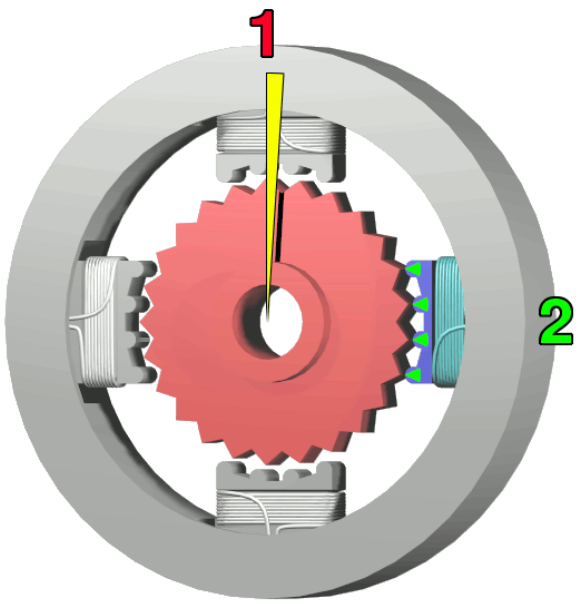
Step Angle

- The step angle is the increment by which a stepper motor moves each time it receives a step pulse.
  - Smaller step angles mean higher resolution, resulting in finer control of motion
  - The resolution of a stepper motor is typically given in steps per revolution, where the number of steps is determined by the motor's construction
  - Formula: Step Angle = 360° / Number of Steps per Revolution
  - **Example**, a motor with 200 steps per revolution has a step angle of 360° / 200 = 1.8° per step

Practical Stepper motor operation

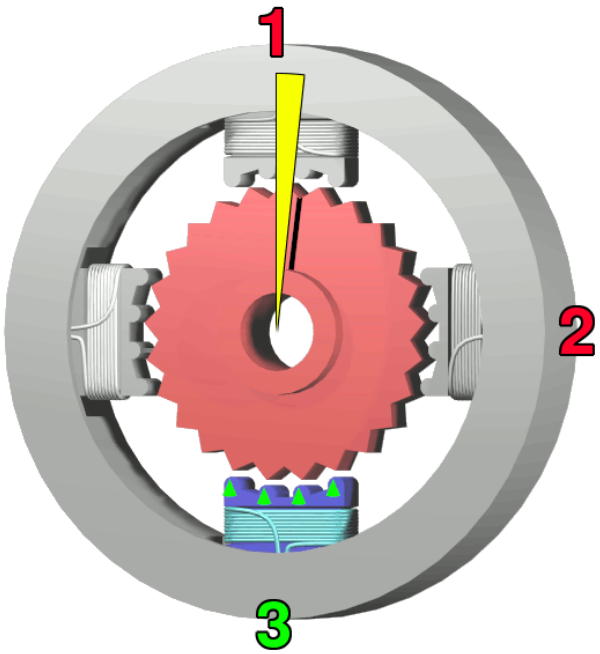


The top electromagnet (1) is turned on, attracting the nearest teeth of a gear-shaped iron rotor. With the teeth aligned to electromagnet 1, they will be slightly offset from electromagnet 2

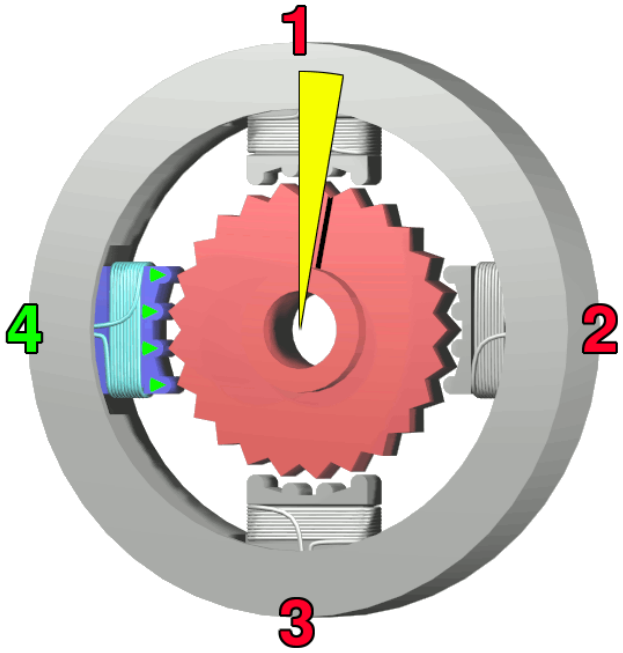


The top electromagnet (1) is turned off, and the right electromagnet (2) is energized, pulling the nearest teeth slightly to the right. This results in a rotation of 3.6° in this example.

Practical Stepper motor operation



The bottom electromagnet (3) is energized; another 3.6° rotation occurs.



The left electromagnet (4) is enabled, rotating again by 3.6°. When the top electromagnet (1) is again enabled, the teeth in the sprocket will have rotated by one tooth position; since there are 25 teeth, it will take 100 steps to make a full rotation in this example.

Stepper Motor Basics

Holding Torque & Torque Speed Characteristics

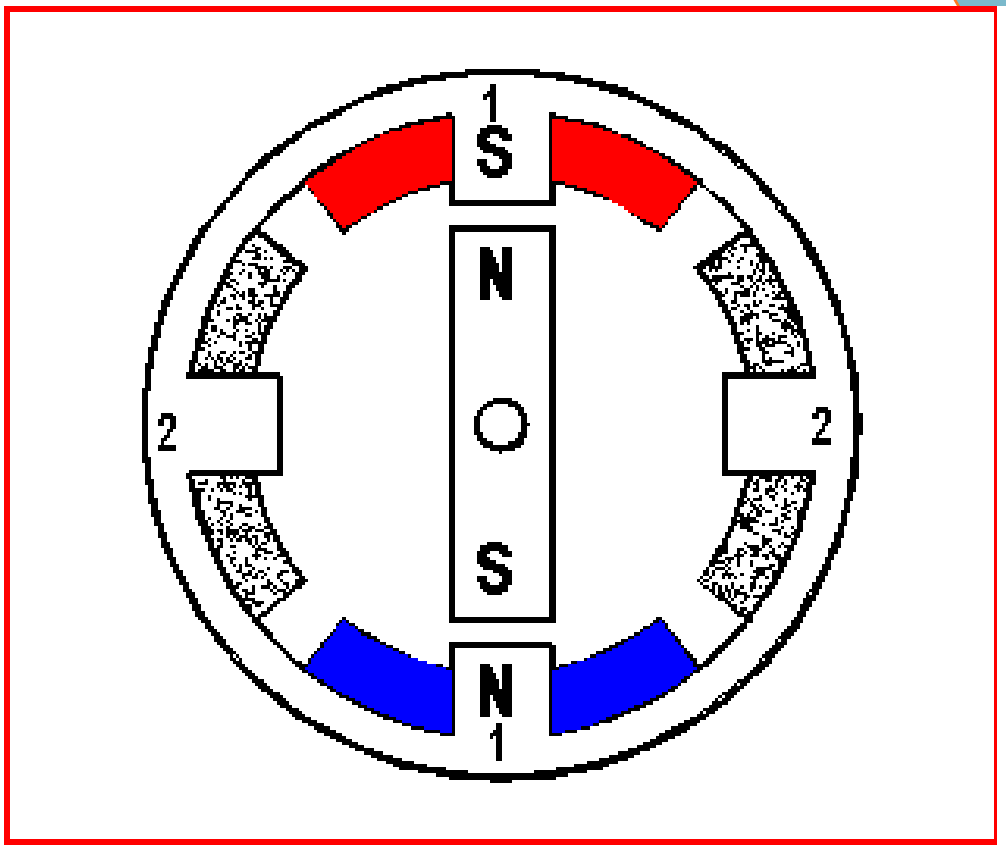
- **Holding Torque:**
  - The torque required to maintain the motor's position when not rotating.
  - An important factor in applications that need the motor to hold its position under load.
- **Torque-Speed Characteristics:**
  - Stepper motors provide high torque at low speeds but lose torque as the speed increases.
  - This characteristic is essential to consider when designing systems that need speed and torque control.

Operating Modes Overview

- 3 Modes
  - Full Step
  - Half Step
  - Microstepping

Full Step Mode

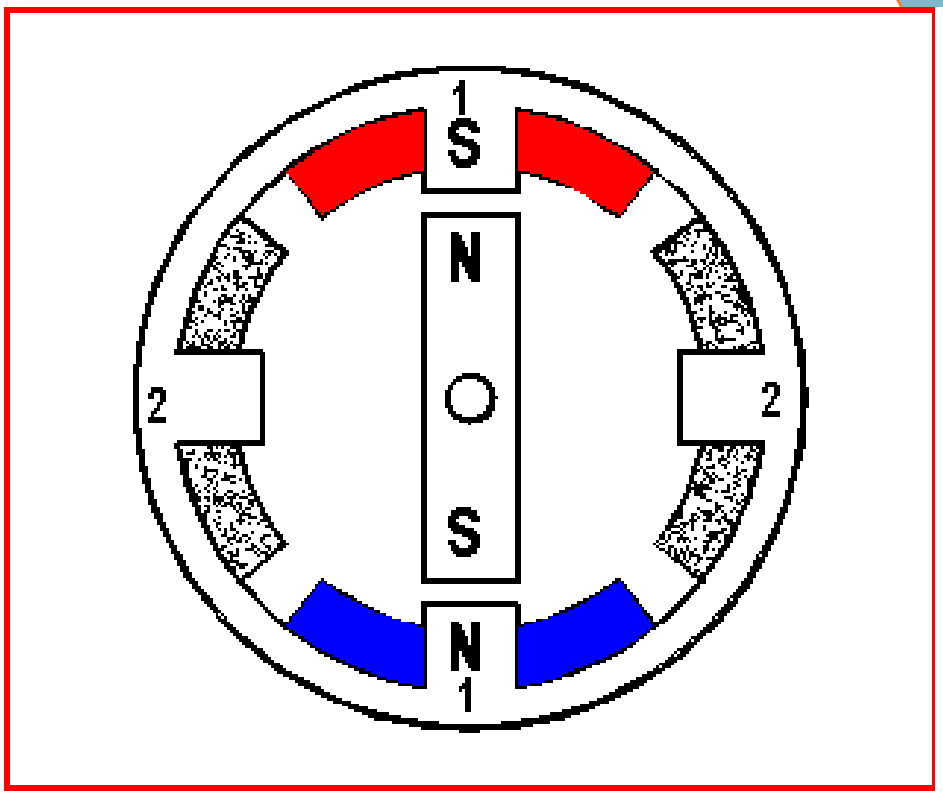
- In full step mode, the stepper motor rotates by its full step angle with each pulse
  - This mode provides the highest torque
  - Each coil is energized in turn resulting in a full step of the rotor,thus, maximizing the magnetic pull on the rotor
  - Suitable for applications where maximum torque is required and resolution is not a critical factor
  - **Example:**
  - A 90° step angle motor moves 90° per step in this mode.



Four Steps per revolution i.e. 90 deg. steps.

Half Step Mode

- Half step mode combines full steps with intermediate steps, effectively doubling the resolution
  - The motor alternates between full step positions and steps halfway in between
  - This mode provides a balance between higher resolution and decent torque
  - It reduces torque ripple, resulting in smoother motion compared to full step mode
  - Example: For a 90° motor, each half step is 45°



Eight steps per. revolution i.e. 45 deg. steps.

Microstepping

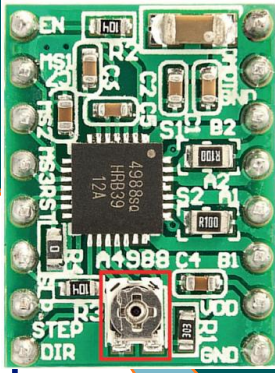
- Microstepping divides each full step into smaller increments by controlling the current levels in the motor windings
  - This mode achieves smoother motion and higher resolution, making it suitable for applications that require precision
  - Reduces vibration and noise during operation
  - Allows for non-integer step angles, providing flexibility in motion control
  - Commonly used for high-precision devices like 3D printers and CNC machines

Controlling Stepper Motors

- Stepper motors can be are generally controlled using dedicated drivers and microcontrollers.
  - Microcontrollers like Arduino and MSP430 can generate step pulses to drive the motor.
  - A4988, DRV8825 are popular stepper motor drivers that simplify control and offer features like microstepping. We will use the A4988 AZ Delivery board in the lab

A4988 Stepper Driver

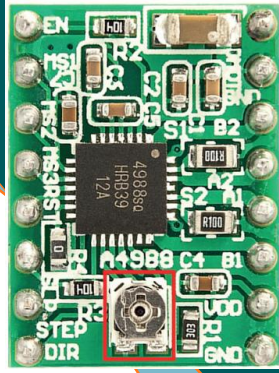
Features



- The A4988 is a popular stepper motor driver used for controlling bipolar stepper motors.
  - Supports microstepping up to 1/16th of a step for smoother motion.
  - Built-in overcurrent and thermal shutdown protection.
  - Adjustable current control to limit the maximum current output.
  - Step and direction interface allows for easy control using microcontrollers.
  - Suitable for applications requiring precise control of stepper motors.
- **NOTE:** A user manual for the A4988 Stepper Driver can be found in the datasheets folder in Blackboard

A4988 Stepper Driver

Basic Connections (Pin Out)



- **VDD and GND**: Power supply for the logic (3.3V or 5V).
- **VMOT** and **GND**: Power supply for the motor (typically 8V to 35V).
- **STEP:** Pin used to control the motor's movement; each pulse corresponds to a step.
- **DIR:** Pin used to set the direction of rotation (clockwise or counterclockwise).
- **ENABLE:** Enables or disables the driver (low to enable, high to disable).
- **MS1, MS2, MS3**: Pins used to set the microstepping mode.

Microstepping Configuration:

A4988

- The A4988 allows for different microstepping settings:
  - Full Step: All MS1, MS2, and MS3 pins are set to low
  - Half Step: MS1 is set to high, and MS2 and MS3 are set to low
  - 1/4 Step: MS2 is set to high, and MS1 and MS3 are set to low
  - 1/8 Step: MS1 and MS2 are set to high, and MS3 is set to low
  - 1/16 Step: MS1, MS2, and MS3 are all set to high.
- Microstepping allows for finer control and smoother motion of the stepper motor

Microstepping Configuration:

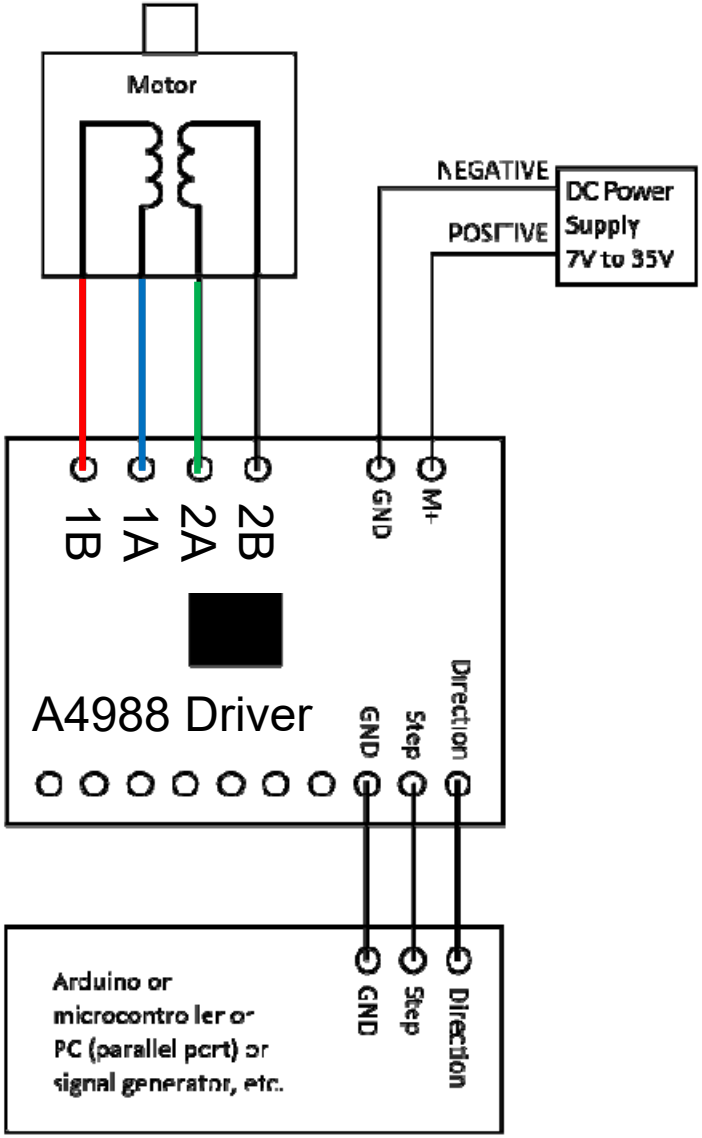
A4988 Driver

By setting suitable logic levels on these pins, we can set the drive mode of the motor to one of these five modes:

MS1	MS2	MS3	Microstep resolution
LOW	LOW	LOW	Full step
HIGH	LOW	LOW	Half step
LOW	HIGH	LOW	Quarter step
HIGH	HIGH	LOW	Step of eight
HIGH	HIGH	HIGH	Sixteenth step

These three microstep pins are switched LOW by internal pull-down resistors so that the motor operates in full step mode when we disconnect them all.

Basic Wiring Diagram



Controlling a Stepper Motor from a Microcontroller

MSP430

```
#include <msp430.h>

void main(void)
{

  WDTCTL = WDTPW + WDTHOLD;  // Stop watchdog timer
  P2DIR=0x03;                // P2.1 (Enable) and P2.0 (Direction) are outputs
  P1DIR=0x01;                //P1.0 (Step) is set up as an output
  TA0CCR0 |= 500-1;          // Setting timer 0 to count 499 Pulses
  TA0CCTL0 |= CCIE;          // Enable Timer Interrupt
  TA0CTL |= 0x????;          // input correct configuration of TASSEL (SMCLCK) and MC1 (Continuous Mode) TA0CTL Bits
  _BIS_SR(GIE);              //Enables GlobalInterrupts


    while(1)
    {
          P2OUT = 0x00;             // Anticlockwise one direction
          __delay_cycles(1000000);
          P2OUT = 0x02;             //off
          __delay_cycles(1000000);
          P2OUT = 0x01;             //other direction
          __delay_cycles(1000000);
          P2OUT = 0x02;             //off
          __delay_cycles(1000000);
    }
}


#pragma vector=TIMER0_A0_VECTOR
   __interrupt void Timer0_A0 (void) {
       P1OUT ^= BIT0;
}
```

Controlling a Stepper Motor from a Microcontroller

MSP430

12.3.1 TACTL Register

Timer_A Control Register

TACTL is shown in Figure 12-16 and described in Table 12-4.

Return to Table 12-3.

Figure 12-16. TACTL Register

15	14	13	12	11	10	9	8
Unused						TASSELx	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
IDx		MCx		Unused	TACLR	TAIE	TAIFG
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Table 12-4. TACTL Register Field Descriptions

Bit	Field	Type	Reset	Description
15-10	Unused	R/W	0h	Unused
9-8	TASSELx	R/W	0h	Timer_A clock source select 00b = TACLK 01b = ACLK 10b = SMCLK 11b = INCLK (INCLK is device-specific and is often assigned to the inverted TBCLK) (see the device-specific data sheet)
7-6	IDx	R/W	0h	Input divider. These bits select the divider for the input clock. 00b = /1 01b = /2 10b = /4 11b = /8
5-4	MCx	R/W	0h	Mode control. Set MCx = 00b when Timer_A is not in use to conserve power. 00b = Stop mode: the timer is halted. 01b = Up mode: the timer counts up to TACCR0. 10b = Continuous mode: the timer counts up to 0FFFFh. 11b = Up/down mode: the timer counts up to TACCR0 then down to 0000h.

Calculating the length of time it takes to Rotate 360°

- TACCR0 |= 500-1; // counter count from 0 – 499 i.e. 500 times.
- Each time the counter reaches 499 it flips BIT0 of P1 (see the interrupt code).
  - Therefore:- creates a square wave with 1KHz frequency on P1.0
    - i.e. 1000 Pulse per second
  - Number of pulse required for full step mode:
    - Step size = 1.8^{o}
    - 360°/1.8^{o} = 200 steps per rotation
    - 16th Step mode = 200 * 16 = 3200 steps per rotation
    - Time to rotate 360° = 3200 /1000. = 3.2 seconds

Drawbacks of the timing Method

- If the timer on the microcontroller is inaccurate then the number of steps completed in that time is inaccurate
- How do we make it more accurate?
- Rather than apply pulses to the step pin for a period of time. We should count the number of steps applied to the microcontroller.

Different Types of Stepper Motor

1. **Permanent Magnet Stepper Motor:**
   - Uses a permanent magnet rotor
   - Provides good torque at low speeds
   - Commonly used in low-power applications

2. **Variable Reluctance Stepper Motor:**
   - Uses a soft iron rotor that is attracted to the stator poles
   - Faster response time but generally provides lower torque

3. **Hybrid Stepper Motor:**
   - Combines features of permanent magnet and variable reluctance stepper motors
   - Offers higher torque and precision compared to other types
   - Commonly used in industrial applications

Stepper Motor Comparison

- Comparison of Permanent Magnet, Variable Reluctance, and Hybrid Stepper Motors:
  - **Step Angle:** Hybrid stepper motors typically have a smaller step angle (higher resolution)
  - **Torque:** Hybrid motors offer the highest torque, followed by permanent magnet and then variable reluctance
  - **Speed:** Variable reluctance stepper motors can achieve higher speeds than the other two types
  - **Applications:** Permanent magnet for low-power tasks, hybrid for high precision, and variable reluctance for speed-sensitive tasks



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Interfacing to Servo Motors

Servo Motors

Introduction

- A Servo is a small motor with an output shaft. This shaft can be positioned to specific angular positions by sending the servo a coded signal
- As long as the coded signal exists on the input line, the servo will maintain the angular position of the shaft
  - As the coded signal changes, the angular position of the shaft changes
- In practice, servos are used in radio controlled airplanes to position control surfaces like the elevators and rudders. They are also used in radio controlled cars, puppets, and of course, robots





Operation/Control of a Servo

- A servo, unlike a simple d.c. motor has three inputs – Vcc, Gnd and Control - a line on which the angular position governing signal is sent
- Controlling the shaft position of a servo involves sending a modulated square-wave pulse which is known as Pulse-Width-Modulation (PWM)
- The servo expects to see pulses every 20 milliseconds (.02 seconds), or at a frequency of 50 Hz (i.e. 50 pulses per second) on its control line



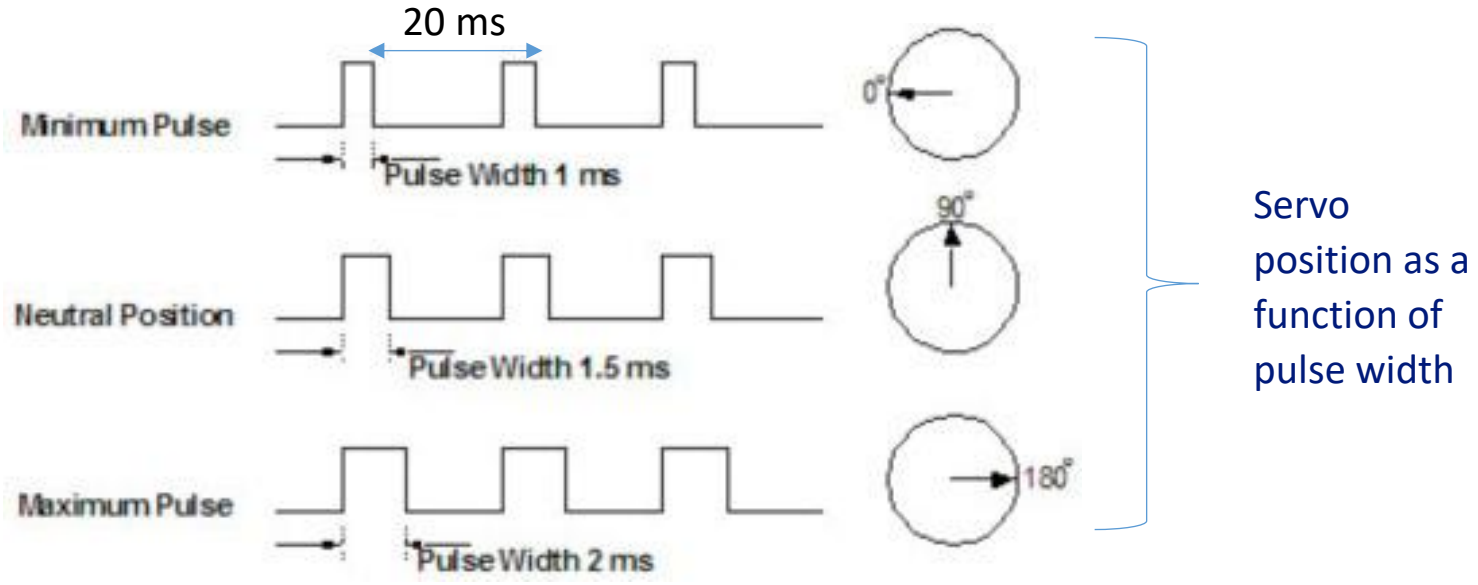
Operation/Control of a Servo

- The width of the pulses determines how far the motor shaft turns.
  - A 1.5 millisecond pulse, for example, will make the motor turn to the 90 degree position(approx). This is often called the neutral position.
  - If the pulse is shorter than 1.5 ms, then the motor will turn the shaft to closer to 0 degrees.
  - If the pulse is longer than 1.5ms, the shaft turns closer to 180 degrees.
  - Maximum pulse width is usually 2ms (approx.)



Operation/Control of a Servo

- The servomotor position control signal sent to the servomotor consists of short pulses (1-2ms in duration) sent every 20ms
- The control signal MUST BE MANTAINED (keep sending the pulses!!!) to keep the motor shaft in a particular position (i.e. at a particular angle)



- **Note:** Servomotors vary from manufacturer to manufacturer, especially in the angle through which they will turn for a given pulse width. The above information is indicative only – consult the data sheet of the servo you intend to use for more detailed information

Controlling a Motor from a Microcontroller

MSP430

- The problem of controlling a servo from a microcontroller is not so much the generation of the pulses but the fact that the pulse train must be produced indefinitely, to maintain the servo position.
- **Approach 1: Simple delays**
  - Eg For a servo attached to P2.0, to be positioned at 90° , the following code could be used (though it would probably require some adjustment):

```
while(1)
        {
P2OUT=P2OUT|0x01;        // High on control line
__delay_cycles(1500) ;   // wait 1.5ms
P2OUT=P2OUT&0xfe;        // Low on control line
__delay_cycles(18500);    // wait 18.5ms
}
```

Controlling a Motor from a Microcontroller

MSP430

```
while(1)
        {
        P2OUT=P2OUT|0x01;        // High on control line
        __delay_cycles(1500) ;   // wait 1.5ms
        P2OUT=P2OUT&0xfe;        // Low on control line
        __delay_cycles(18500);    // wait 18.5ms
        }
```

- **The code poses several problems:**

  i. The microcontroller must maintain the servo pulses at all times – it is NOT FREE to perform any other tasks, except servo control.

  ii. The ***__delay_cycles(xx)*** function only accepts literal constants i.e. we cannot, at least using this programming structure, obtain variable pulse i.e. variable shaft positions.

Controlling a Servo Motor from a Microcontroller

MSP430 Interrupt Based Approach

```
#include <msp430g2553.h>// Servo Motor control using Timers/Interrupts
unsigned int counter = 0; // Servo counter
int servoPosition0=1500; // value 1000 to 2000 gives pulse width of 1 to 2ms
void main(void)
{
  // int i;
    WDTCTL = WDTPW + WDTHOLD;              // Stop WDT
    BCSCTL1 = CALBC1_1MHZ;   // 1MHz Clock
    DCOCTL = CALDCO_1MHZ;
    P2OUT = P2OUT&0xfc;
    P2DIR =P2DIR|0x03;          // Port P2.0 and P2.1 servo outputs
    P2SEL = P2SEL&0xfc;
    P1DIR=0xff;
    P1OUT=0xff;
    // Set up timer and enable timer interrupts
    TA0CCTL0 = 0x0010;          // CCR0 interrupt enabled
    TA0CCR0 = 2;
    TA0CTL = 0x0211;        // SMCLK, upmode
    __enable_interrupt(); // Enable interrupt
   // Timer interrupt will happen every TA0CCR0 microseconds
   // 1 millisecond = 1000 microseconds

    while(1)
    {  // main function code
      // do any required tasks here
    //  to change the servo position just assign a new value to servoPosition0
//  (between 1000 and 2000)

 }
}
```

```
// Timer A0 interrupt service routine
#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A0(void)
{   counter++;
    P2OUT=P2OUT&0xfe;              // Clear all servo control port pins
    if(counter==1)     // Do Pulse?
    {
      P2OUT = P2OUT|0x01;
      TA0CCR0 = servoPosition0;// Set time for servo
    }
      else
    {// do remainder of 20ms
     TA0CCR0 = 20000 - servoPosition0;
     counter=0;
    }
}
```

Code Explanation

- **The code operates as follows:**
  - The system clock is fixed at 1MHz and the watchdog timer is disabled (stopped)
  - Port 2 is configured as an output on P2.0 and P2.1
  - The Timer interrupt is enabled/configured and set to occur every TA0CCR0 microseconds where TA0CCR0 is the contents of the 16-bit TA0CCR0 register (0 - 65,535). TA0CCR0 is initially set to 2 so that a TIMER interrupt occurs almost immediately
- The Timer interrupt service routine (ISR) is written so that the first time it executes (and on all subsequent odd-numbered executions) it sets the servo control output P2.0 HIGH, and sets the TIMER interrupt to occur next after servoPosition0 microseconds Having servoPosition0 set to 1500 for example gives a pulse of 1.5ms
- The 2nd (and all even numbered) execution of the ISR resets the servo control pin P2.0 LOW and sets the interrupt to occur when the full 20ms cycle of servomotor control has elapsed

Summary

- **What is a Stepper motor?**
  - Why use them, typical Applications
  - Advantages / Disadvantages
- **Different Types of Stepper Motor**
- **Interfacing with Stepper Motors**
  - Drivers
  - Code for interfacing and control (MSP430)
- **How stepper motors operate.**
  - Different Modes of Operation
- **What are Servo Motors?**
  - Operation and Control
  - Code for interfacing and control (MSP430)



Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University